Future Improvements and Optimizations for Mountain NER

The current Named Entity Recognition pipeline successfully leverages a custom-headed **distilbert-base-cased** model fine-tuned on a hybrid dataset (human-annotated Kaggle records + LLM-generated synthetic data) using AWS SageMaker.

Deployed via a containerized FastAPI web service, the system achieves highly efficient inference latencies. However, as with any v1.0 machine learning system, there are opportunities to improve model precision, training robustness, and deployment scalability.

1. Data-Centric Improvements

- **Hard Negative Mining:** Inference edge case testing revealed a false-positive bias where the model misclassified non-geographical capitalized entities (e.g., "Apple Store" in New York) as mountains. Future synthetic data generation should explicitly include "hard negatives" (e.g., brands, standard cities, organizations) labeled strictly as **0(Outside)**. This forces the model to learn the semantic difference between commercial and geographical entities.
- **Active Learning Pipeline:** Implement a feedback loop in the API to log low-confidence predictions. These edge-case sentences can be routed to a human annotator and periodically injected back into the training dataset.

2. Architectural & Modeling Upgrades

- **Integration of a CRF Layer**: Currently, the model uses a standard Linear classification head with a Cross-Entropy loss function. Adding a Conditional Random Field (CRF) layer on top of the network would enforce strict BIO transition rules. A CRF learns that an I-MOUNTAIN tag can mathematically never follow an O tag, instantly eliminating structurally impossible predictions.
- **Transfer Learning from a Base NER Model**: Instead of fine-tuning the raw language model (distilbert-base-cased), initialization could start from a model already trained on the CoNLL-03 dataset (e.g., dslim/bert-base-NER). This provides the network with foundational knowledge of generic Organizations and Locations, allowing it to filter out irrelevant entities easily before applying the specialized mountain classification.

3. Deployment & MLOps Scaling

- **Model Quantization**: To further optimize the FastAPI Docker container, the PyTorch model weights can be converted to ONNX format and INT8 quantized. This strips away unnecessary 32-bit floating-point precision, shrinking the container footprint and dropping CPU inference latency even further.
- **Batch Inference Endpoint**: The current /predict endpoint processes a single text query at a time. Expanding the Pydantic schema to accept a List[str] will allow the endpoint to leverage dynamic batching, drastically increasing the API's throughput for bulk processing requests.